

MERN E-COMMERCE APPLICATION

Phase 1: Project Setup

1. Project Structure

- Folder architecture
 - Environment variables
 - Configuration
 - Path aliases
 - Constants
 - Utilities
 - Error handling
 - Logging
-

2. Backend Setup

- Express
- TypeScript (optional)
- MongoDB
- Mongoose
- Helmet
- CORS
- Compression
- Cookie Parser
- Morgan/Pino Logger
- Rate Limiter
- dotenv

3. Frontend Setup

- React
- Vite
- TypeScript
- React Router
- Axios
- React Query / TanStack Query
- Redux Toolkit / Zustand
- Tailwind CSS
- React Hook Form
- Zod

Phase 2: Authentication

Registration

- User registration
- Validation
- Email normalization
- Password hashing
- Duplicate checking

Login

- JWT
- Refresh Token
- HttpOnly Cookies
- Account lock

- Login history
-

Authorization

- User
- Admin
- Super Admin

Role middleware

Permission middleware

Account

- Verify Email
 - Resend Verification
 - Forgot Password
 - Reset Password
 - Change Password
 - Change Email
 - Delete Account
-

Phase 3: User Module

Profile

- View Profile
 - Edit Profile
 - Avatar Upload
 - Address Management
 - Phone Number
-

Address

- Add Address
 - Edit Address
 - Delete Address
 - Default Address
-

Wishlist

- Add
 - Remove
 - Move to Cart
-

Phase 4: Product Module

Categories

CRUD

Nested Categories

Brands

CRUD

Products

Create

Read

Update

Delete

Soft Delete

Restore

Product Fields

- Name
- Description
- Price
- Discount
- Images
- Category
- Brand
- SKU
- Stock
- Variants
- Rating

Product Images

- Upload
- Delete
- Reorder

Cloudinary

Variants

Size

Color

Weight

Storage

RAM

Phase 5: Search

- Search
 - Autocomplete
 - Suggestions
-

Filters

Price

Category

Brand

Rating

Availability

Discount

Color

Size

Sorting

Newest

Price Low

Price High

Popularity

Rating

Phase 6: Cart

Add

Update Quantity

Delete

Clear Cart

Guest Cart

Merge Guest Cart

Phase 7: Wishlist

Add

Remove

Move to Cart

Phase 8: Checkout

Address

Shipping

Payment

Coupon

Review

Confirmation

Phase 9: Payment

Stripe

Razorpay

Cash on Delivery

Webhook

Payment Verification

Refund

Phase 10: Orders

Create Order

View Orders

Cancel Order

Return Request

Invoice

Order Timeline

Order Status

Pending

Confirmed

Packed

Shipped

Out For Delivery

Delivered

Cancelled

Returned

Refunded

Phase 11: Shipping

Shipping Cost

Tracking

Courier

Delivery Date

Phase 12: Reviews

Add Review

Update

Delete

Like Review

Report Review

Phase 13: Coupons

Create

Update

Delete

Expiry

Usage Limit

Minimum Order

Maximum Discount

Phase 14: Notifications

Email

Push

In App

SMS

Phase 15: Admin Dashboard

Dashboard

Statistics

Charts

Revenue

Orders

Products

Users

Sales

Phase 16: User Management

View Users

Ban User

Suspend User

Delete User

Role Management

Phase 17: Inventory

Stock

Low Stock

Out Of Stock

Inventory History

Phase 18: Analytics

Revenue

Top Products

Top Customers

Monthly Sales

Order Reports

Phase 19: Reports

Sales

Users

Orders

Refunds

Inventory

 Phase 20: Security

Helmet

CORS

Rate Limiting

XSS

CSRF

Input Validation

Sanitization

JWT

Cookies

Password Hashing

Audit Logs

 Phase 21: Performance

Caching

Redis

Lazy Loading

Pagination

Image Optimization

Compression

Indexes

Aggregation

Phase 22: File Upload

Cloudinary

Image Resize

Delete

Multiple Upload

Phase 23: Email

Welcome

Verification

Reset Password

Order Confirmation

Shipping Update

Invoice

Phase 24: Responsive UI

Desktop

Tablet

Mobile

Accessibility

Phase 25: UI Components

Navbar

Sidebar

Footer

Cards

Buttons

Forms

Modal

Toast

Pagination

Breadcrumb

Carousel

Skeleton Loading

Phase 26: API

REST

Validation

Error Handling

Versioning

Swagger/OpenAPI Documentation

Phase 27: Testing

Unit Tests

Integration Tests

API Tests

Component Tests

Phase 28: Deployment

Backend

Frontend

MongoDB Atlas

Cloudinary

Environment Variables

Domain

HTTPS

CI/CD



Phase 29: Documentation

README

API Documentation

Setup Guide

Architecture Diagram

ER Diagram

Deployment Guide



Bonus Features

- AI Product Recommendations
- Recently Viewed Products
- Compare Products
- Product Recommendations
- Frequently Bought Together
- Multi-language
- Dark Mode
- Multi-currency
- Seller Dashboard (Marketplace)

- Live Chat
 - Chatbot
 - Order Tracking Map
 - Invoice PDF
 - QR Code Orders
 - WebSockets for Live Notifications
-

Professional Folder Structure

ecommerce/

|

└─ client/

|

└─ components/

|

└─ pages/

|

└─ hooks/

|

└─ services/

|

└─ store/

|

└─ utils/

|

└─ layouts/

|

└─ server/

|

└─ controllers/

|

└─ services/

|

└─ repositories/

|

└─ middleware/

|

└─ validators/

| ├── models/
| ├── routes/
| ├── utils/
| ├── config/
| ├── jobs/
| ├── emails/
| └── uploads/
|
└── docs/

Interview-Ready Checklist

By the end of this project, you should be able to explain:

- Authentication & Authorization
- JWT vs Sessions
- Password Hashing (bcrypt/Argon2)
- Cookies & Security
- REST API Design
- MongoDB Schema Design
- Database Indexing
- Aggregation Pipelines
- Pagination & Filtering
- File Uploads
- Payment Integration
- State Management
- Performance Optimization
- Caching

- Security Best Practices
 - Deployment
 - CI/CD Basics
-

My suggestion

Don't try to build all 29 phases at once. Build them incrementally.

A strong portfolio project that gets attention from recruiters usually includes:

1. Authentication
2. Product catalog
3. Cart
4. Checkout
5. Payment integration
6. Orders
7. Admin dashboard
8. Product management
9. User management
10. Security and deployment

Once these are solid and well-documented, you already have a project that demonstrates most of the skills expected from a junior to mid-level MERN developer.

Absolutely. In fact, **this is exactly how software companies manage projects**. Every feature/module has **Entry Criteria, Tasks, Exit Criteria, Testing, and Definition of Done (DoD)**.

For a portfolio project, adding exit criteria is one of the best ways to ensure each phase is genuinely complete before moving on.

For example:

Phase	Entry Criteria	Exit Criteria
Authentication	Express, MongoDB, User model ready	User can register, login, logout, refresh token works, protected routes work, password reset works, email verification works, all tests pass

Instead of just saying "Authentication complete", you'll know exactly what "complete" means.

Example: Authentication Module

Entry Criteria

- Express project created
- MongoDB connected
- User schema created
- Environment variables configured

Tasks

- Registration
- Login
- JWT
- Refresh Token
- Logout
- Forgot Password
- Email Verification
- Rate Limiting

Exit Criteria

Functional

-  User registration works

- ☒ Login works
- ☒ Logout works
- ☒ Refresh token works
- ☒ Forgot password works
- ☒ Reset password works
- ☒ Verify email works
- ☒ Protected routes work
- ☒ Role middleware works

Security

- ☒ Password hashed
- ☒ JWT expires
- ☒ Refresh token stored securely
- ☒ Cookies are HttpOnly
- ☒ Rate limiter enabled
- ☒ Helmet enabled
- ☒ CORS configured
- ☒ No sensitive data returned

Testing

- ☒ Invalid password tested
- ☒ Duplicate email tested
- ☒ Duplicate username tested
- ☒ Locked account tested
- ☒ Expired token tested
- ☒ Invalid token tested

Documentation

-  API documented
-  README updated

Definition of Done

- No console errors
 - No warnings
 - All endpoints tested
 - Postman collection updated
 - Code committed
-

Example: Product Module

Entry Criteria

Authentication complete

Tasks

- Product CRUD
 - Category CRUD
 - Brand CRUD
 - Image Upload
 - Variants
-

Exit Criteria

Functional

- Create Product
- Update Product
- Delete Product

- Restore Product
- View Products
- Pagination
- Search
- Filter
- Sorting

Security

- Only Admin can Create
- Only Admin can Delete
- Validation works

Performance

- Pagination implemented
- Database indexed
- Images optimized

Testing

- CRUD tested
- Upload tested
- Validation tested

Documentation

- Product API documented

Example: Payment Module

Exit Criteria

Functional

- Stripe Payment
- Razorpay Payment

- COD
- Webhook
- Payment Verification
- Refund

Security

- Webhook Signature Verified
- Payment Amount Verified
- Duplicate Payment Prevented

Testing

- Successful Payment
- Failed Payment
- Cancelled Payment
- Refund

Professional Checklist

For **every module**, use the same template:

Module Name

Objective

Prerequisites (Entry Criteria)

Requirements

Implementation Tasks

Database Changes

API Endpoints

Frontend Pages

Security Checklist

Validation Rules

Error Handling

Testing Checklist

Performance Checklist

Documentation

Exit Criteria

Definition of Done

Future Improvements

What I can create for you

I can prepare a **complete Software Engineering Roadmap PDF** for a **production-level MERN E-commerce application**, where every module includes:

- 🎯 Objective
- 📖 Concepts to Learn
- 🛠️ Technologies Used
- 📋 Prerequisites (Entry Criteria)
- 📝 Detailed Task List
- 🗄️ Database Schema
- 🔌 API Endpoints
- 🎨 Frontend Pages
- 🧩 Components
- 🛡️ Security Checklist
- ⚠️ Validation Rules
- 🧪 Testing Checklist
- 🚀 Performance Checklist
- ✅ Exit Criteria
- 🏁 Definition of Done
- ★ Interview Questions
- 📁 Real-world Industry Tips
- ⌚ Estimated Time
- 📊 Difficulty (Beginner / Intermediate / Advanced)